

HireProof.tech Audit – SEO, Performance & Roadmap

Summary

HireProof.tech provides a novel tool for detecting recruitment scams and offers a polished interface with strong branding. The site uses structured data (JSON-LD) for **Organization**, **Person**, **WebSite** and **SoftwareApplication**, has meaningful meta-descriptions, keywords and Open-Graph/Twitter tags in its HTML head, and the canonical URL points to `hireproof.tech` rather than `www` ¹. A large amount of content is served through Next.js client components, including a complex nav bar, interactive scanning demos, a research lab and animated footers. These enhance user engagement but cause **heavy JavaScript bundles** and degrade Core Web Vitals, especially **Interaction to Next Paint (INP)**. A phased plan below prioritizes SEO fixes, reduction of client-side work and progressive enhancement.

Current strengths

- **Meaningful meta tags & JSON-LD** – The `<head>` includes meta description, keywords, canonical link and structured data for company and app ¹, giving search engines context.
- **Scam-detection tool & demos** – The **LinkedIn demo** page uses animated progress logs and risk scores to demonstrate the extension ². The **lab** page streams logs and updates step statuses while consuming an audit API ³.
- **Rich UI/UX** – Vibrant hero sections, dark/light themes, tooltips, and responsive layouts show strong design skills.
- **Structured navigation & footer** – Links are grouped by product, resources and legal categories, and each item includes descriptive text in the code ⁴. A `ScrollToTopControl` provides quick navigation ⁵.

Issues & opportunities

Area	Findings & impact
Domain & canonical	The canonical tag references <code>https://hireproof.tech</code> ¹ , but there is no 301 redirect to enforce the preferred URL. <code>www.hireproof.tech</code> should permanently redirect to <code>hireproof.tech</code> to prevent duplicate indexing and consolidate link equity.
Huge client navigation	The header (<code>site-header.tsx</code>) is a client component with imports of <code>usePathname</code> , <code>useState</code> , <code>useRef</code> and <code>useEffect</code> and manages menu open/close, pointer events, focus trapping and theme toggling. It hydrates on every page and includes dropdowns and search modals, which inflates the JS bundle and delays first interaction.

Area	Findings & impact
Heavy demos	The LinkedIn demo page uses multiple <code>useState</code> hooks, <code>useEffect</code> timers, and Framer Motion to mimic scanning and risk scoring ² . This page loads large assets and scripts even if users just browse; it should be isolated and lazily loaded.
Complex lab client	The <code>LabClient</code> component orchestrates an audit workflow: it tracks steps, logs, progress, timers and consumes a streaming API ³ . Running audits on the client increases JS load and complexity; unused code ships to all users.
Framer Motion & tooltips	Many components import Framer Motion and <code>AnimatePresence</code> for subtle animations. For example, the footer's tooltip uses <code>useState</code> , timeouts and animated <code><motion.div></code> to show descriptions ⁶ . While visually appealing, these animations add ~20KB+ to the bundle.
Scroll-to-top & other UI controls	<code>ScrollToTopControl</code> adds an effect listener and toggles visibility on scroll ⁵ . These small features compound across the app; they could be implemented with CSS or loaded only when needed.
JavaScript-only routing	Navigation links rely on <code>usePathname</code> and <code>Link</code> but don't degrade gracefully if JS is disabled. Search engines can crawl Next.js routes, but unnecessary client hydration can hurt SEO and accessibility.
Font & asset loading	Fonts are loaded using CSS and may block rendering. Images and icons are loaded eagerly; there is no evidence of <code>next/font</code> or <code>next/image</code> for optimal delivery.

Recommended roadmap

Mindset: Focus on measurable gains – improving Web Vitals and crawling is more impactful than adding more animations. Treat each phase as a **main quest**; finish it before starting decorative work.

Phase 1 – Foundation & SEO

- Permanent redirect** – Configure a 301 redirect from `www.hireproof.tech` to `hireproof.tech` (and ensure HTTP→HTTPS) to consolidate signals. Keep the canonical pointing to the apex domain ¹.
- Meta & structured data audit** – Verify each route has a unique `<title>` and meta description; avoid using the same tags across pages. Continue using JSON-LD for **Organization**, **Person** and **SoftwareApplication** but add **BreadcrumbList** for docs and labs. Use absolute URLs in `@id` fields.
- Robots & sitemap** – Generate a dynamic `sitemap.xml` and `robots.txt` via Next.js API route. Include all important pages (audit engine, explore, docs, pricing, developer portal) and exclude private/dynamic demo routes. Submit the sitemap to Google Search Console.
- Link architecture** – Consolidate thin or duplicate pages; avoid index bloat by combining the various demo pages under a single `/demo` hub. Ensure internal links use descriptive anchor text (e.g., “Audit Engine – verify suspicious jobs” rather than generic “Audit”).

Phase 2 – Navigation & common layout

1. **Convert header to a server component** – Move the `SiteHeader` to a server component that renders static HTML and CSS. Only the **theme toggle** and **search/command menu** should be client islands loaded via `next/dynamic`. Precompute active link states on the server using the current URL; avoid `usePathname` in most cases. This will shrink the JS bundle significantly.
2. **Reduce dropdown complexity** – Replace hover-activated dropdowns with accessible, CSS-only menus where possible. For mobile, use the built-in `<details>` element; this reduces custom JS for focus management.
3. **Simplify `ScrollToTopControl`** – Use CSS `scroll-behavior: smooth` and a simple anchor link to the top rather than stateful visibility logic. If still needed, wrap it in a dynamic import so it does not load on pages that are short.
4. **Streamline footer** – The footer currently uses tooltips animated with Framer Motion ⁶. Replace these with `title` attributes or CSS-based tooltips; load the heavy `AnimatePresence` component only on interactive pages (e.g., docs). Pre-render the health check as static or fetch it on the server via `getServerSideProps`.

Phase 3 – Demo & lab refactoring

1. **Isolate demos behind separate routes** – Move `app/demo/linkedin/page.tsx` and other demonstration pages into a `/demo` subdirectory and mark them as client components. Use `next/dynamic` with `ssr: false` so these pages are loaded only when a user intentionally navigates to them. Do not include their scripts in the main bundle.
2. **Reduce animation libraries** – For the LinkedIn demo, use simple CSS keyframes for fade-ins instead of importing Framer Motion. Limit progress log updates; update the DOM fewer times to avoid main-thread thrashing. Provide a “skip animation” button to jump directly to the result.
3. **Move the lab workflow server-side** – The `LabClient` component handles the entire audit workflow on the client ³. Migrate this logic to an API route and handle stream parsing server-side. On the client, show a `<form>` that posts to the API and returns a full report; if progressive streaming is desired, use the native `<form>` with the `replace` attribute or `SSE` in a Web Worker to avoid blocking the main thread. This will improve INP and reduce user CPU usage.
4. **Cancel & reset logic** – Keep cancel and reset functions, but implement them using standard `<button type="reset">` semantics and server-side state. Avoid storing large arrays of logs in state; render only the last 20 entries to reduce memory usage.

Phase 4 – Asset & performance optimisation

1. **Load fonts with `next/font`** – Use Next.js' built-in font optimization (`next/font/google`) to download only necessary font subsets and display fallback fonts while loading. Eliminate blocking CSS imports.
2. **Optimize images and icons** – Replace inline SVG icons imported from `lucide-react` with `next/image` and `next/font` icons or a sprite sheet. Use `priority` for hero images and `loading="lazy"` for below-the-fold content. Preload only critical assets in the `<head>`.
3. **Defer non-critical scripts** – Use `defer` on analytics scripts and load chat or third-party widgets after `load` event. Avoid `dangerouslySetInnerHTML` for injecting large JSON; instead read config on the server and serve minimal script tags.

4. **Enable HTTP caching** – Set long-term cache headers for static assets (fonts, images, CSS) and implement a service worker if offline access is needed. Use [Next.js incremental static regeneration](#) to cache audit results and blog pages.

Phase 5 – Continuous monitoring & experimentation

1. **Install Vercel Speed Insights or Lighthouse CI** – Continuously measure **LCP**, **FID/INP** and **CLS** after each change. Set thresholds (e.g., <2.5 s LCP, <200 ms INP) and track improvements.
2. **A/B test simplified UI** – Run experiments with simplified navigation vs. current design to quantify drop-off rates and conversion changes. Use this data to decide whether to keep or remove certain interactive features.
3. **Progressive enhancement** – Ensure the site renders core content without JavaScript. For example, the audit tool should fall back to an HTML `<form>` if JS fails. Progressive enhancement benefits SEO and improves accessibility.

Conclusion

HireProof.tech successfully markets a scam-detection tool through rich demos and storytelling. However, the site's reliance on heavy client-side components and animations harms performance and could hurt search rankings. By following the phased plan above—starting with domain canonicalization, simplifying shared components, isolating demos, and optimizing assets—the team can achieve faster load times, improved **Core Web Vitals**, and better crawlability while retaining a polished user experience. Each phase should be treated as a discrete mission: execute it decisively, measure the impact, and resist the temptation to add new features until the foundational work is complete.

1 HireProof | Verify Job Posts Before Applying

<https://hireproof.tech/>

2 raw.githubusercontent.com

<https://raw.githubusercontent.com/Iron-Mark/Hackathon-HireProof/ce4ff32f292e7959eff3bda141702ad46175c006/app/demo/linkedin/page.tsx>

3 raw.githubusercontent.com

<https://raw.githubusercontent.com/Iron-Mark/Hackathon-HireProof/ce4ff32f292e7959eff3bda141702ad46175c006/app/lab/lab-client.tsx>

4 6 raw.githubusercontent.com

<https://raw.githubusercontent.com/Iron-Mark/Hackathon-HireProof/ce4ff32f292e7959eff3bda141702ad46175c006/components/layout/site-footer.tsx>

5 raw.githubusercontent.com

<https://raw.githubusercontent.com/Iron-Mark/Hackathon-HireProof/ce4ff32f292e7959eff3bda141702ad46175c006/components/layout/scroll-to-top-control.tsx>